

**A Report**  
**On the Project work done during semester - V subject**  
**Advanced Technologies**  
**(CE – 520)**  
**of**  
**B.Tech. Computer Engineering**  
**SurveyKaro.com**

|                                                                                                                                                        |                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prepared by:</b><br><b>Smit Bhansali</b><br><b>22CEUON114</b><br><b>CE-068</b><br><b>Computer Engineering,</b><br><b>Dharmsinh Desai University</b> | <b>Prepared by:</b><br><b>Saif Huseni</b><br><b>22CEUON112</b><br><b>CE-073</b><br><b>Computer Engineering,</b><br><b>Dharmsinh Desai University</b> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

**Guided By - Prof. P. M. Jadav**



**Faculty Of technology**  
**Dharmsinh Desai University, Nadiad**  
**October 2024**

# Dharmsinh Desai University

College Road, NADIAD – 387001. (Gujarat)



## Certificate

This is to certify that the term work carried out in the subject of **(CE520) Advanced Technologies** and submitted is the bonafide work of **Mr. Smit Ajaybhai Bhansali, Mr. Saif Sahil Huseni** Roll No.: **CE068, CE073** Identity No.: **22CEUON114, 22CEUON112** of B.Tech. **Semester V** in the branch of **Computer Engineering** during the academic year 2024-2025.

Prof. P. M. Jadav  
Associate Professor

Dr. C. K. Bhensdadia  
Head, CE Department

# Table of Contents

---

| Sr. No | Title                                     | Page |
|--------|-------------------------------------------|------|
| 1      | Abstract                                  | 4    |
| 2      | Introduction                              | 5    |
| 3      | System analysis                           | 7    |
| 4      | Software Requirements Specification (SRS) | 9    |
| 5      | Database Design                           | 18   |
| 6      | Implementation Detail                     | 22   |
| 7      | Testing                                   | 28   |
| 8      | Screenshots                               | 32   |
| 9      | Conclusion                                | 36   |
| 10     | Limitation & Future Implementation        | 37   |
| 11     | Bibliography                              | 39   |

# 1. Abstract

---

SurveyKaro.com is a web-based platform developed using the MERN (MongoDB, Express.js, React, Node.js) stack, designed to streamline the process of creating, managing, and responding to surveys. The application allows users to register and securely log in using JWT-based authentication, ensuring that each user's data is protected. Once logged in, users can create custom surveys with various question types, distribute them to participants, and analyze the responses in real time.

The system is built with a focus on simplicity, efficiency, and user-friendly interaction, making it easy for both survey creators and respondents to engage with. On the backend, MongoDB serves as the database, storing surveys, responses, and user profiles. The project also features robust API endpoints to manage survey operations such as creation, submission, and retrieval of results.

The goal of SurveyKaro.com is to provide a flexible and comprehensive solution for gathering feedback, whether for academic, business, or personal purposes. The platform enhances the experience of data collection and offers insights into user responses in an intuitive and interactive manner.

## 2. Introduction

---

### 2.1 Introduction

In today's fast-paced digital world, gathering feedback is crucial for making informed decisions, whether in business, education, or research. Surveys are an effective tool for collecting and analyzing data from various stakeholders. However, creating, distributing, and managing surveys can be cumbersome without the right platform. SurveyKaro.com was developed to address this need, providing an online solution that simplifies the survey process.

The platform allows users to log in securely, create customizable surveys, distribute them to respondents, and analyze the feedback in real time. Designed with simplicity and ease of use in mind, SurveyKaro.com provides a user-friendly interface for both survey creators and participants, streamlining the feedback collection process. This project highlights the use of modern web development technologies to create an efficient and scalable solution for feedback management.

### 2.2 Technology/Platform/Tools Used

This combination of technologies ensures that *SurveyKaro.com* is fast, scalable, and user-friendly, making it suitable for both small-scale and large-scale survey projects.

- **Frontend:** The user interface is built using **React(^18.13.1)**, a popular JavaScript library that allows for efficient and reusable component-based UI development.
- **Backend:** The server-side functionality is handled by **Node.js(^20.15.1)** and **Express.js(^4.21.0)**, enabling fast and scalable **API development** to manage surveys, users, and responses.

- **Database: MongoDB(^8.5.4)** serves as the database, storing survey data, user profiles, and survey responses in a flexible and scalable manner. **MongoDB Compass** was used for managing and viewing data.
- **Authentication:** User authentication is secured using **JWT (JSON Web Tokens)( ^9.0.2)**, providing a reliable way to ensure secure login and session management.
- **Development Tools: Postman** was used to test and validate API routes, while **Visual Studio Code** was the primary IDE for development. Version control was managed through **Git and GitHub**.

## 3. System analysis

### # Use Case Diagram

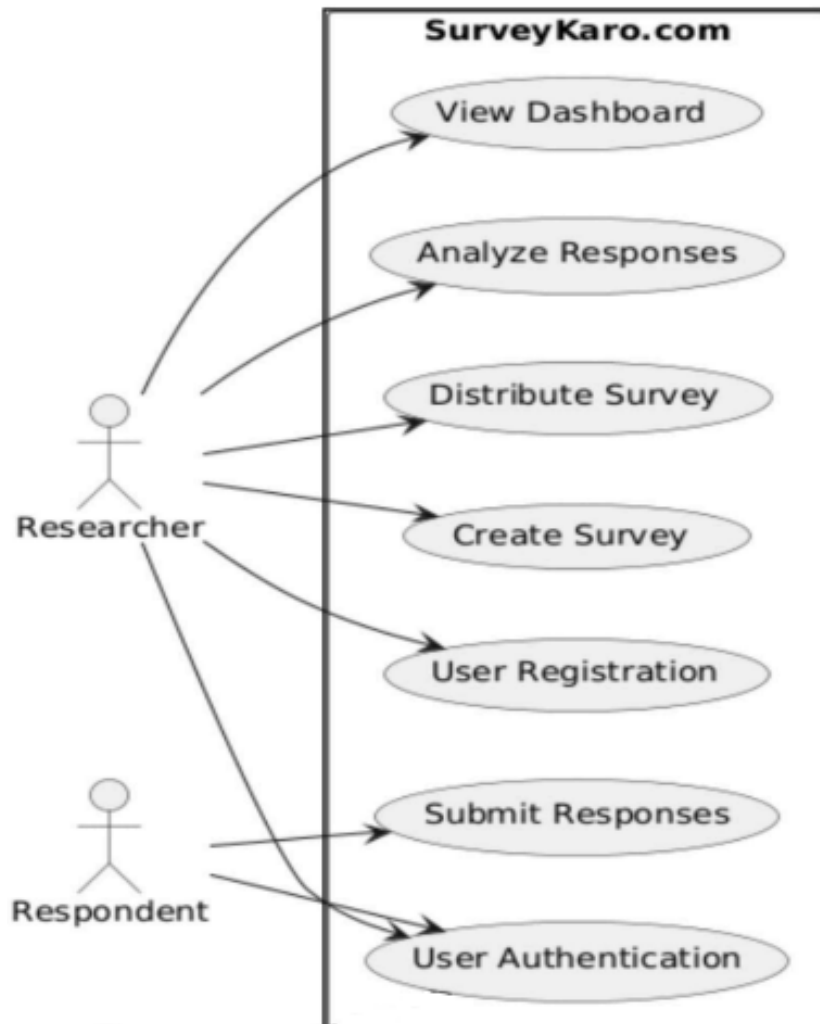
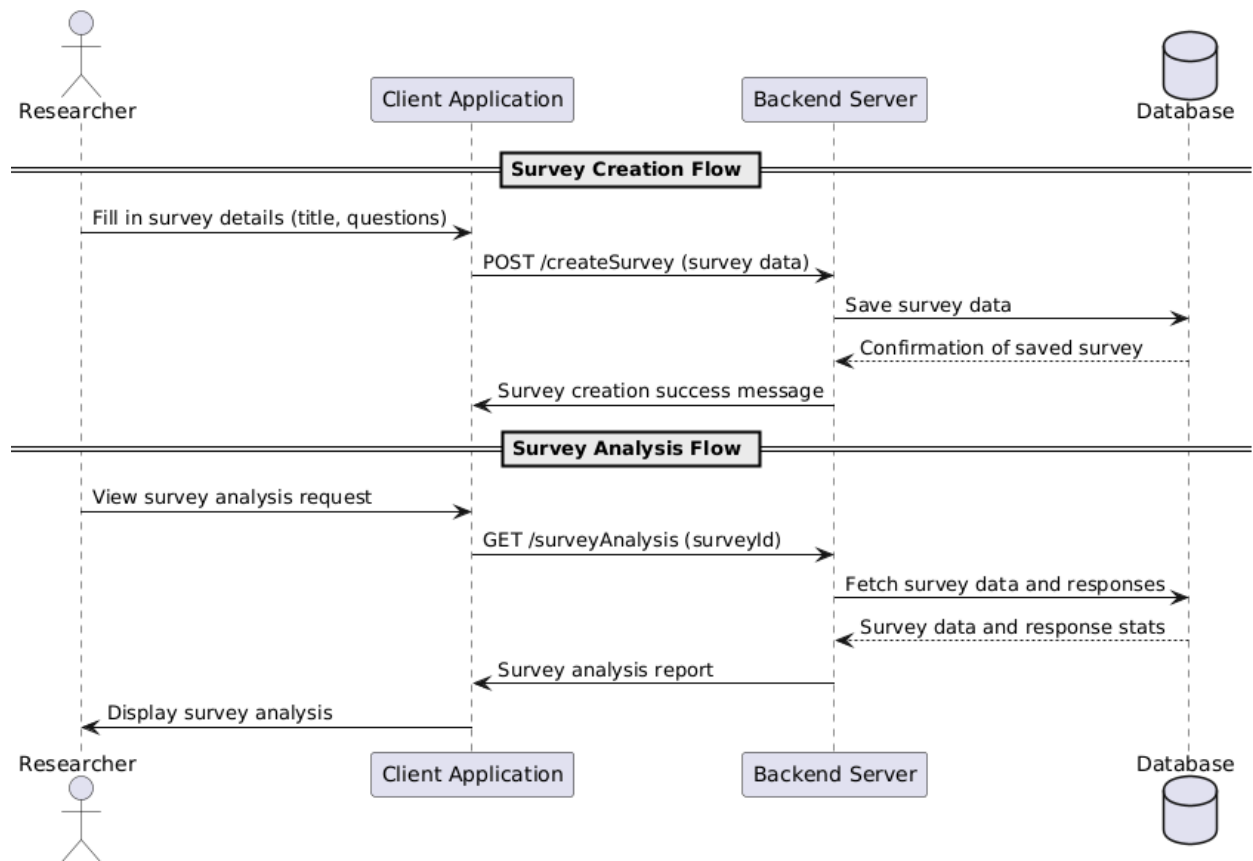


Fig 3.1 Use-Case diagram

## # Sequence Diagram:



**Fig 3.2 Sequence diagram**



# 4. Software Requirements Specification (SRS)

---

## 1. Introduction

### 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to outline the functional and non-functional requirements for the development of an online feedback and survey system. This document aims to provide a comprehensive understanding of the system's goals and features for both the development team and stakeholders.

### 1.2 Document Conventions

This document is created based on the IEEE template for System Requirement Specification Documents.

### 1.3 Intended Audience and Reading Suggestions

The intended audience for this document includes:

- Development Team: Review the functional and non-functional requirements thoroughly to gain a clear understanding.
- Stakeholders: Including project managers, product owners, and investors should focus on the overall description, system features, and goals of implementation to align with the team's objectives.
- Quality Assurance Team: Review the entire SRS, especially the functional and non-functional requirements sections, to create the best possible test case scenarios.
- End Users: Familiarize themselves with the capabilities of the feedback and survey system.

## **1.4 Product Scope**

The product scope includes the development of a feedback and survey system that allows researchers to create and distribute various types of surveys, including single-choice, multiple-choice, subjective questions, rating scales, and agreement scales. The system will provide researchers with insights through visual charts such as bar graphs and pie charts.

## **2. Overall Description**

### **2.1 Product Perspective**

The feedback and survey system will operate as a web-based platform, enabling researchers to create, distribute, and analyze surveys and feedback forms. The platform will support standard web protocols and be accessible on various browsers and devices.

### **2.2 Product Functions**

- User Registration and Authentication: Users will create personalized accounts and log in securely.
- Survey and Feedback Creation: Researchers can create various types of questions, including single-choice, multiple-choice, subjective, rating scales, and agreement scales.
- Response Analysis and Visualization: Researchers can analyze responses using visual charts such as bar graphs and pie charts.
- User Dashboard: Users have personalized dashboards to manage their activities, including creating surveys, viewing responses, and analyzing data.

### **2.3 User Classes and Characteristics**

- Researchers: Users who create surveys and feedback forms, analyze responses, and gain insights.
- Respondents: Users who participate in surveys and provide feedback.

- Administrators: Manage user accounts, monitor activities, and ensure system security and compliance.
- System Developers: Build and maintain the system, address technical issues, and implement new features.

## **2.4 Operating Environment**

The feedback and survey system will be available on various web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. It will require a stable internet connection and be compatible with multiple operating systems, including Windows, macOS, and Linux.

## **2.5 Design and Implementation Constraints**

- The system will be developed using the MERN stack (MongoDB, Express.js, React.js, Node.js).
- The interface will primarily be in English.
- The choice of the database should support high data traffic and quick querying.
- The system should be compatible with HTML5 and modern web standards.

## **2.6 User Documentation**

Documentation will be developed for each user class, specifying their rights and terms of use. It will include guides for creating surveys, managing responses, and using the system's features.

## **2.7 Assumptions and Dependencies**

- It is assumed that users will access the system with up-to-date hardware and software.
- The system depends on users having stable internet connections and compatible web browsers.

### 3. External Interface Requirements

#### 3.1 User Interfaces

- Registration and Authentication Interface: For users to create profiles and log in securely.
- Survey Creation Interface: For researchers to create and manage surveys.
- Response Analysis Interface: For researchers to visualize and analyze survey responses.
- User Dashboard Interface: To manage user activities and provide an overview of surveys and responses.

#### 3.2 Hardware Interfaces

The system will support common user devices, including desktops, laptops, tablets, and mobile devices with good internet connectivity and compatible web browsers.

#### 3.3 Software Interfaces

- Backend: Developed using **Node.js** and **Express.js** to handle server-side operations and communicate with the database.
- Frontend: Developed using **React.js** to provide a responsive and interactive user interface.
- Database: **MongoDB** will be used to store user data, survey questions, and responses.

#### 3.4 Communications Interfaces

The system might use third-party services for secure data transmission and storage. It will also employ notification services to keep users informed about survey statuses and updates.

## **4. System Features**

### **4.1 User Registration and Authentication**

- Users can create personalized accounts with secure authentication.
- The system ensures secure login and data protection.

### **4.2 Survey and Feedback Creation**

- Researchers can create various types of questions, including single-choice, multiple-choice, subjective, rating scales, and agreement scales.
- Surveys can be distributed to respondents via links or email invitations.

### **4.3 Response Analysis and Visualization**

- Researchers can view responses in real-time.
- The system provides visual charts such as bar graphs and pie charts for data analysis.

### **4.4 User Dashboard**

- Users have personalized dashboards to manage their activities.
- The dashboard displays an overview of surveys, responses, and analytical insights.

## **5. Functional Requirements**

### **->Login**

#### **R.1: User Authentication**

- Description: The system shall allow users to log in using their registered credentials.
- R.1.1: Enter Username and Password
  - Input: User provides a valid username and password.
  - Output: System accepts credentials for verification.
- R.1.2: Verify Credentials
  - Input: System checks entered username and password.
  - Output: Successful verification or denial.
- R.1.3: Handle Invalid Credentials
  - Input: User provides incorrect username or password.
  - Output: System displays an error message and prompts for correct credentials.

### **->Survey Creation**

#### **R.2: Survey Creation**

- Description: Researchers can create surveys with various types of questions.
- R.2.1: Create Single-Choice Question
  - Input: Researcher selects single-choice question type and provides options.
  - Output: Question is added to the survey.
- R.2.2: Create Multiple-Choice Question
  - Input: Researcher selects multiple-choice question type and provides options.
  - Output: Question is added to the survey.
- R.2.3: Create Subjective Question
  - Input: Researcher selects subjective question type and provides a prompt.
  - Output: Question is added to the survey.

- R.2.4: Create Rating Scale Question
  - Input: Researcher selects rating scale question type and sets scale parameters.
  - Output: Question is added to the survey.
- R.2.5: Create Agreement Scale Question
  - Input: Researcher selects agreement scale question type and provides options.
  - Output: Question is added to the survey.

### **->Response Analysis**

#### R.3: Response Analysis

- Description: Researchers can analyze survey responses using visual charts.
- R.3.1: View Bar Graph
  - Input: Researcher selects bar graph option.
  - Output: System displays a bar graph of responses.
- R.3.2: View Pie Chart
  - Input: Researcher selects pie chart option.
  - Output: System displays a pie chart of responses.
- R.3.3: Export Data
  - Input: Researcher selects export data option.
  - Output: System exports survey data in CSV format.

### **->Survey Submission**

#### R.4: Survey Submission

- Description: Normal website users/Respondents

## **6. Other Nonfunctional Requirements**

### **6.1 Performance Requirements**

- Response Time: The system shall maintain a response time of under 5 seconds for key user interactions.
- Scalability: The system must scale to accommodate a growing user base without compromising performance.

### **6.2 Safety Requirements**

Data Security: The application shall employ robust encryption protocols to ensure the secure transmission and storage of user data.

User Authentication: The application will implement multi-factor authentication to enhance account security.

### **6.3 Security Requirements**

- Data Backup: Regular automated backups of the application's database shall be performed to safeguard against data loss.

### **6.4 Software Quality Attributes**

Usability: The application shall maintain an intuitive and user-friendly interface.

Reliability: The system shall operate with a 99.9% uptime, minimizing disruptions to user access and survey activities.

### **6.5 Business Rules**

Compliance: The application must comply with relevant data protection and privacy regulations.



## Goal of Implementation :

- 1. User-Centric Design:** Develop an intuitive and visually appealing interface for seamless user experiences.
- 2. Scalability and Performance:** Design a scalable architecture to handle growing user traffic and survey activities while maintaining optimal system performance.
- 3. Security and Trust:** Implement robust security measures to safeguard user data and interactions.
- 4. Efficient Survey Processes:** Streamline survey creation and response analysis processes for efficiency and transparency.
- 5. Continuous Improvement:** Establish a framework for continuous improvement, incorporating user feedback and system performance monitoring.

## References :

IEEE System Requirements Specification Template

<https://react.dev/> React.js Documentation

<https://nodejs.org/en> Node.js Documentation

<https://expressjs.com/> Express.js Documentation

<https://docs.mongodb.com/> MongoDB Documentation

## 5. Database Design

---

In **SurveyKaro.com**, the database is designed using **MongoDB**, a NoSQL database that offers flexible, document-based storage. The system is built around three core entities: **Users**, **Surveys**, and **Survey Responses**. These entities are modeled in MongoDB using collections and documents that efficiently manage the relationships and structure of the data.

MongoDB's flexibility allows for embedding related documents within each other where appropriate, reducing the need for complex joins and ensuring fast read operations for the most frequent queries.

### 4.1 Collections Overview

The following collections are used in the database:

- **Users**: Stores user account information.
- **Surveys**: Stores survey metadata and questions.
- **SurveyResponses**: Stores user-submitted responses to survey questions.

### 4.2 Collection

#### a. Users Collection

The **Users** collection is responsible for storing user information such as usernames, email addresses, and hashed passwords. It also manages user authentication.

### + Key Points :

- The passwords are hashed using **bcrypt** to ensure security.
- Timestamps for **createdAt** and **updatedAt** are automatically generated by Mongoose.

### + Schema :

```
{
  _id: ObjectId("..."),    // Automatically generated unique identifier for each user
  username: String,        // User's unique username
  email: String,           // User's unique email address
  password: String,        // Encrypted password stored securely
  createdAt: ISODate("..."), // Date the user was created
  updatedAt: ISODate("...") // Date the user was last updated
}
```

## b. Surveys Collection

The **Surveys** collection stores information about surveys created by users, including the survey title, description, questions, and the user who created the survey.

### + Key Points:

- **Embedded documents** are used for questions inside each survey.
- Each question has a **questionType**, allowing flexibility for different types of questions such as single-choice, multiple-choice, and rating scale questions.

**+ Schema :**

```

{
  _id: ObjectId("..."),      // Survey ID

  title: String,              // Title of the survey

  description: String,        // Brief description of the survey

  questions: [
    {
      questionType: String, // Type of question (e.g., single-choice, multiple-choice)

      questionText: String,  // The question itself

      options: [String]      // List of possible answers for multiple-choice questions
    }
  ],

  createdBy: ObjectId("..."), // Reference to the user who created the survey

  createdAt: ISODate("..."),  // Date the survey was created

  updatedAt: ISODate("...")   // Date the survey was last updated
}

```

**c. SurveyResponses Collection**

The **SurveyResponses** collection stores the answers provided by users in response to survey questions. Each response is linked to a particular survey and user.

+ **Key Points:**

- **Answers** are stored as an array of responses, with each answer referencing the corresponding questionId.
- Each response is linked to the survey and user, ensuring traceability.

+ **Schema :**

```
{
  _id: ObjectId("..."),      // Unique identifier for the response
  surveyId: ObjectId("..."),  // Reference to the associated survey
  answers: [
    {
      questionId: ObjectId("..."), // Reference to the question being answered
      answer: String                // User's response (varies based on the question type)
    }
  ],
  respondent: ObjectId("..."), // Reference to the user who responded
  createdAt: ISODate("...")    // Date the response was submitted
}
```

## 6. Implementation Details

---

### A) Modules Created and Brief Description of Each Module

#### 1. User Management Module

- **Description:** This module handles user authentication and authorization. It allows users to sign up, log in, and manage their accounts. User passwords are encrypted using bcrypt before being stored in the database.
- **Input Parameters:** Fullname, Email, Password
- **Output Parameters:** JWT token
- **Key Features:**
  - User Registration
  - User Login with encrypted password verification
  - JWT-based Authentication for secure routes
  - Storing and managing user information such as username and email.
- **Relevant Files:**
  - `User.js`: Contains the Mongoose schema for the `User` model.
  - `authController.js`: Contains logic for user registration, login, and authentication.

#### 2. Survey Creation Module

- **Description:** This module allows users to create surveys with different types of questions such as single-choice, multiple-choice, subjective, and rating-scale questions. The survey data is stored in the database, and users can manage the surveys they create.
- **Input Parameters:** JWT token, Survey Details
- **Output Parameters:** Survey(with unique Id)

- **Key Features:**
  - Create and edit surveys.
  - Add different types of questions (single-choice, multiple-choice, subjective, rating-scale).
  - Save surveys in MongoDB with timestamp and user reference.
- **Relevant Files:**
  - **Survey.js**: Contains the Mongoose schema for the **Survey** model, including embedded questions.
  - **surveyController.js**: Handles the creation and editing of surveys.

### 3. Survey Response Module

- **Description:** This module allows users to submit responses to surveys. Responses are linked to the specific survey and user who submitted them. It handles capturing answers based on the question type and storing them in the database.
- **Input Parameters:** JWT token, Survey Id
- **Output Parameters:** Submit confirmation
- **Key Features:**
  - Submit responses for surveys.
  - Capture various types of answers (text, multiple choices, ratings).
  - Store user responses in MongoDB and associate them with the corresponding survey and user.
- **Relevant Files:**
  - **SurveyResponses.js**: Contains the Mongoose schema for the **SurveyResponse** model.
  - **responseController.js**: Handles the submission and storage of survey responses.

## 4. Survey Viewing and Analytics Module

- **Description:** This module allows users to view existing surveys and their corresponding responses. It also generates basic analytics on the survey responses, like calculating the distribution of answers for multiple-choice questions.
- **Input Parameters:** JWT token, Survey Id
- **Output Parameters:** Survey Analysis Report
- **Key Features:**
  - View survey questions and details.
  - Retrieve and display submitted responses.
  - Calculate and display statistics based on submitted responses.
- **Relevant Files:**
  - **surveyController.js:** Retrieves and displays surveys and responses.
  - **SurveyResult.js:** Provides basic analytics on survey results.

## B) Function Prototypes Implementing Major Functionality

### 1. User Authentication

```
// Register a new user

async function registerUser(username, email, password) {

    // Hash the password and store the user in the database

}


// Login a user
```



```
async function loginUser(email, password) {  
    // Check user credentials, generate and return JWT token  
}  
  
// Compare passwords for login  
  
UserSchema.methods.comparePassword = async function  
(candidatePassword) {  
    // Compare hashed password with the stored password  
}
```

## 2. Survey Creation

```
// Create a new survey  
  
async function createSurvey(title, description, questions,  
createdBy) {  
    // Save survey to the database with associated user  
}  
  
// Add a question to the survey  
  
async function addQuestion(surveyId, questionType,  
questionText, options, scale) {  
    // Add a question to an existing survey
```

```
}
```

```
// Fetch all surveys created by a specific user
```

```
async function getUserSurveys(userId) {
```

```
    // Retrieve all surveys from the database for the given  
    user
```

```
}
```

### **3. Survey Response Submission**

```
// Submit a response to a survey
```

```
async function submitSurveyResponse(surveyId, answers,  
    respondentId) {
```

```
    // Save survey response with associated respondent and  
    survey
```

```
}
```

```
// Fetch all responses for a given survey
```

```
async function getSurveyResponses(surveyId) {
```

```
    // Retrieve all responses for a specific survey
```

```
}
```

#### 4. Analytics

```
// Calculate response distribution for a multiple-choice
question

function calculateResponseDistribution(surveyId, questionId)
{
    // Iterate through responses and calculate how many users
    chose each option
}

// Generate analytics for a survey

async function generateSurveyAnalytics(surveyId) {
    // Fetch all responses and calculate basic stats (e.g.,
    average rating, choice distribution)
}
```

# 7. Testing

Table 7.1 Test cases

| Test Case ID | Test Case Description                          | Test Steps                 | Test Data               | Expected Result                                   | Actual Result    | Status |
|--------------|------------------------------------------------|----------------------------|-------------------------|---------------------------------------------------|------------------|--------|
| TU01         | Check user login with valid email and password | 1. Go to login page        | Email: user@example.com | User should be able to login                      | Login successful | Pass   |
|              |                                                | 2. Enter email             | Password: Password123   |                                                   |                  |        |
|              |                                                | 3. Enter password          |                         |                                                   |                  |        |
|              |                                                | 4. Click submit            |                         |                                                   |                  |        |
| TU02         | Check user login with incorrect password       | 1. Go to login page        | Email: user@example.com | Login should fail                                 | Login failed     | Pass   |
|              |                                                | 2. Enter email             | Password: WrongPassword |                                                   |                  |        |
|              |                                                | 3. Enter wrong password    |                         |                                                   |                  |        |
|              |                                                | 4. Click submit            |                         |                                                   |                  |        |
| TU03         | Check user login with empty email              | 1. Go to login page        | Email: (empty)          | Error message should be displayed for empty email | Error message    | Pass   |
|              |                                                | 2. Leave email field empty | Password: Password123   |                                                   |                  |        |
|              |                                                | 3. Enter password          |                         |                                                   |                  |        |

|             |                                                          |                              |                         |                                                        |                         |      |
|-------------|----------------------------------------------------------|------------------------------|-------------------------|--------------------------------------------------------|-------------------------|------|
|             |                                                          | 4. Click submit              |                         |                                                        |                         |      |
| <b>TU04</b> | Check user can log out successfully                      | 1. Log in                    | Email: user@example.com | User should be logged out and redirected to login page | Logged out              | Pass |
|             |                                                          | 2. Click on logout           | Password: Password123   |                                                        |                         |      |
| <b>TU05</b> | Check user registration with valid data                  | 1. Go to registration page   | Username: NewUser       | User should be registered and logged in                | Registration successful | Pass |
|             |                                                          | 2. Fill form with valid data | Email: new@example.com  |                                                        |                         |      |
|             |                                                          | 3. Click submit              | Password: Pass123       |                                                        |                         |      |
| <b>TU06</b> | Check user registration with an already registered email | 1. Go to registration page   | Email: user@example.com | Error message should be displayed for duplicate email  | Error message           | Pass |
|             |                                                          | 2. Enter already used email  |                         |                                                        |                         |      |
|             |                                                          | 3. Click submit              |                         |                                                        |                         |      |

|             |                                         |                              |                                  |                                       |                |      |
|-------------|-----------------------------------------|------------------------------|----------------------------------|---------------------------------------|----------------|------|
| <b>TU07</b> | Check survey creation with valid inputs | 1. Go to create survey page  | Title: Survey Title              | Survey should be successfully created | Survey created | Pass |
|             |                                         | 2. Enter title and questions | Description : survey Description |                                       |                |      |
|             |                                         | 3.create survey              | Question 1: Valid Question       |                                       |                |      |
|             |                                         |                              |                                  |                                       |                |      |

|             |                                                    |                             |                |                                                     |               |      |
|-------------|----------------------------------------------------|-----------------------------|----------------|-----------------------------------------------------|---------------|------|
|             |                                                    |                             |                |                                                     |               |      |
| <b>TU08</b> | Check survey creation with missing required fields | 1. Go to create survey page | Title: (empty) | Error message should be displayed for missing title | Error message | Pass |
|             |                                                    | 2. Leave title field empty  |                |                                                     |               |      |
|             |                                                    | 3. Submit survey            |                |                                                     |               |      |

|             |                                                     |                         |                                          |                                                  |                    |      |
|-------------|-----------------------------------------------------|-------------------------|------------------------------------------|--------------------------------------------------|--------------------|------|
| <b>TU09</b> | Check survey response submission with valid answers | 1. Open survey          | Answers: Valid answers for all questions | Survey response should be submitted successfully | Response submitted | Pass |
|             |                                                     | 2. Answer all questions |                                          |                                                  |                    |      |
|             |                                                     | 3. Submit responses     |                                          |                                                  |                    |      |

|             |                                                       |                                  |                             |                                                          |                                         |      |
|-------------|-------------------------------------------------------|----------------------------------|-----------------------------|----------------------------------------------------------|-----------------------------------------|------|
| <b>TU10</b> | Check survey response submission with missing answers | 1. Open survey                   | Answers: Missing one answer | Error message should be displayed for incomplete answers | Error message                           | Pass |
|             |                                                       | 2. Leave one question unanswered |                             |                                                          |                                         |      |
|             |                                                       | 3. Submit responses              |                             |                                                          |                                         |      |
| <b>TU11</b> | Check survey responses viewed by the survey creator   | 1. Log in as survey creator      | Survey: Existing survey     | Survey responses should be displayed with analysis       | Responses displayed along with analysis | Pass |

|             |                                             |                                              |                         |                                       |                 |      |
|-------------|---------------------------------------------|----------------------------------------------|-------------------------|---------------------------------------|-----------------|------|
|             |                                             | 2. View survey result                        |                         |                                       |                 |      |
|             |                                             |                                              |                         |                                       |                 |      |
| <b>TU12</b> | Respond to a valid survey                   | 1.enter valid survey code in the dashboard   | valid survey code       | survey should be shown                | survey is shown | pass |
| <b>TU13</b> | Respond to a invalid survey                 | 1.enter invalid survey code in the dashboard | invalid survey code     | error should be shown                 | error           | pass |
| <b>TU14</b> | Check survey deletion by the survey creator | 1. Log in as survey creator                  | Survey: Existing survey | Survey should be successfully deleted | Survey deleted  | Pass |
|             |                                             | 2. Delete an existing survey                 |                         |                                       |                 |      |

## 8. Screenshots

### 8.1 Homepage

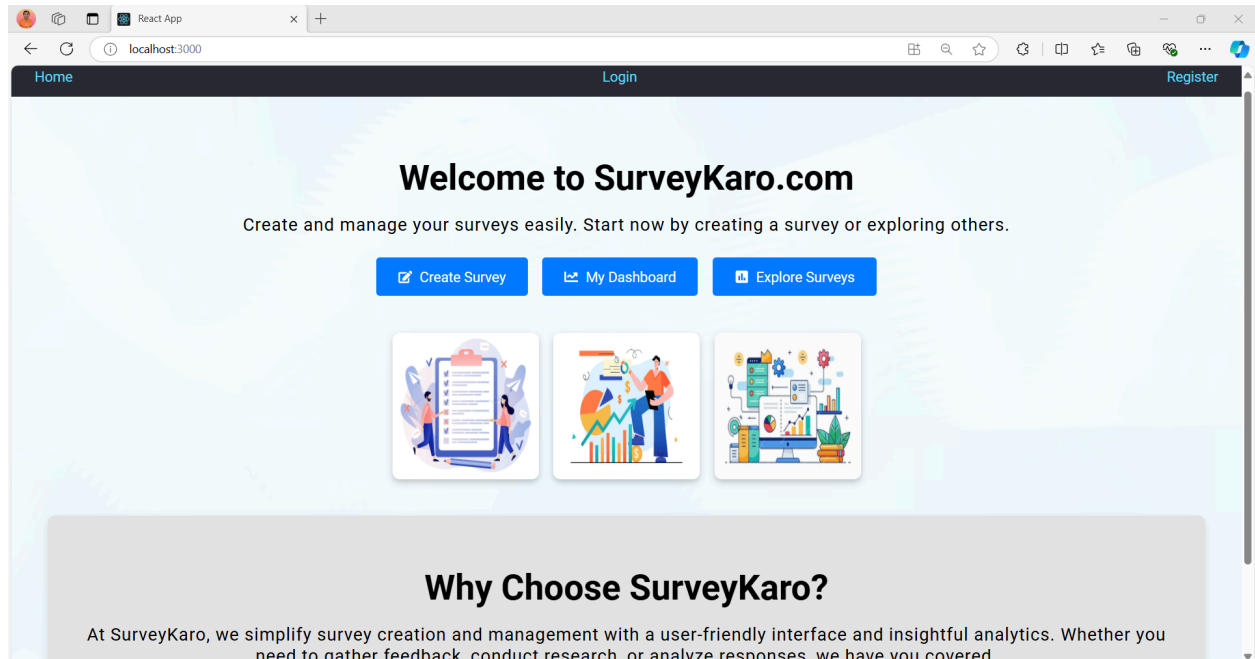


Fig 8.1 Homepage

### 8.2 Loginpage

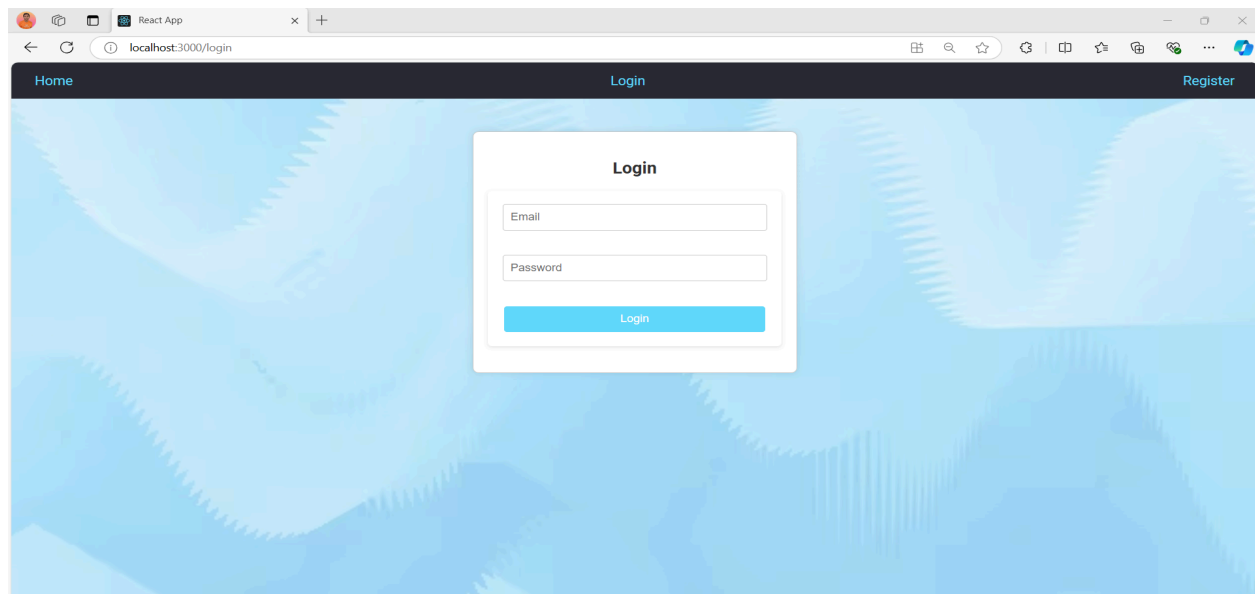


Fig 8.2 login Page



## 8.3 Dashboard

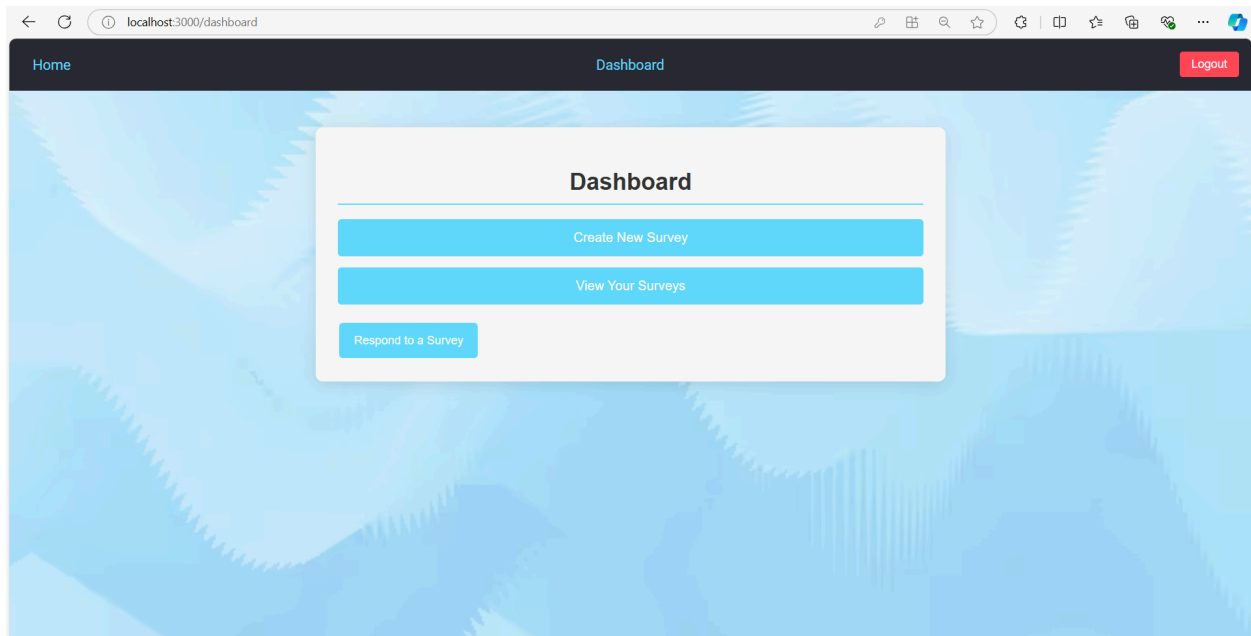


Fig 8.3 Dashboard

## 8.4 Create Survey page

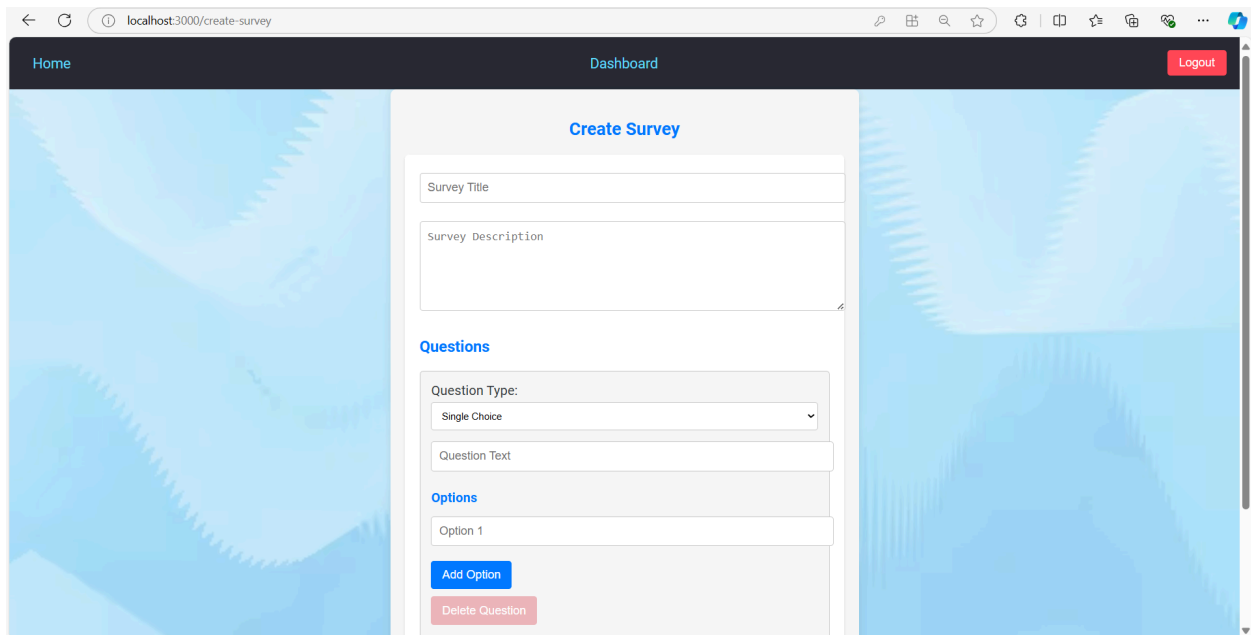


Fig 8.4 create survey

## 8.5 Update Survey page

Home Dashboard Logout

### Update Survey

Title:  
Customer Feedback Survey

Description:  
Survey to gather customer feedback.

#### Questions

Question Type:  
Single Choice

Question Text:  
What is your preferred product?

#### Options

Product A

Product B

Fig 8.5 update survey

## 8.6 Survey Analysis page

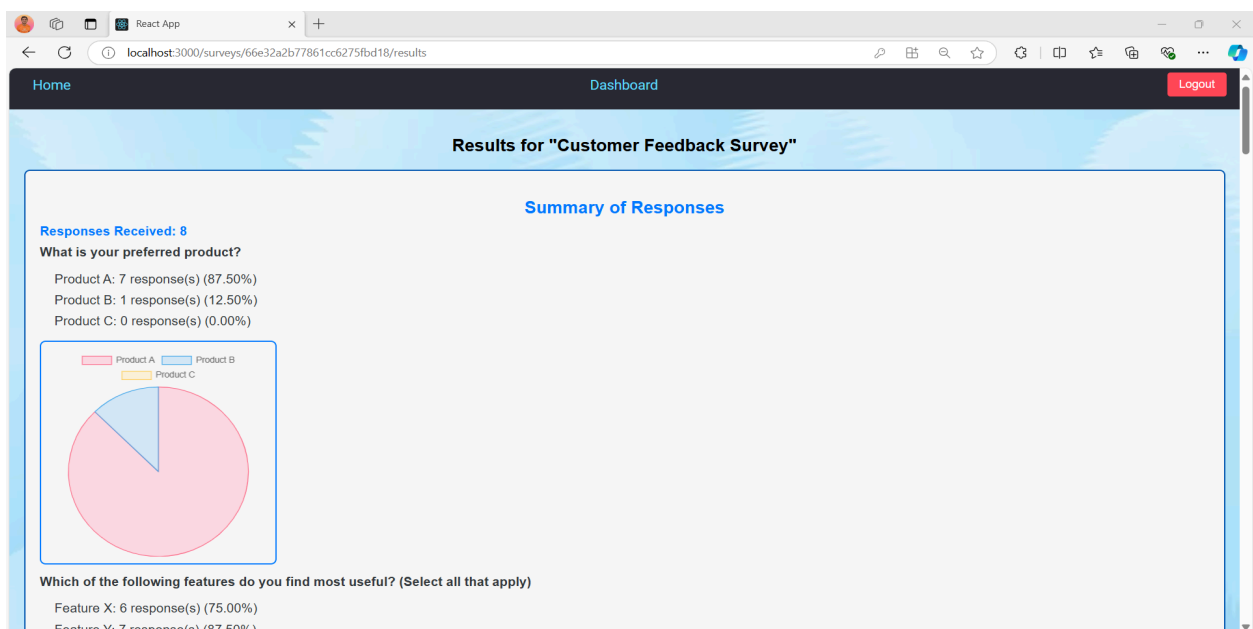
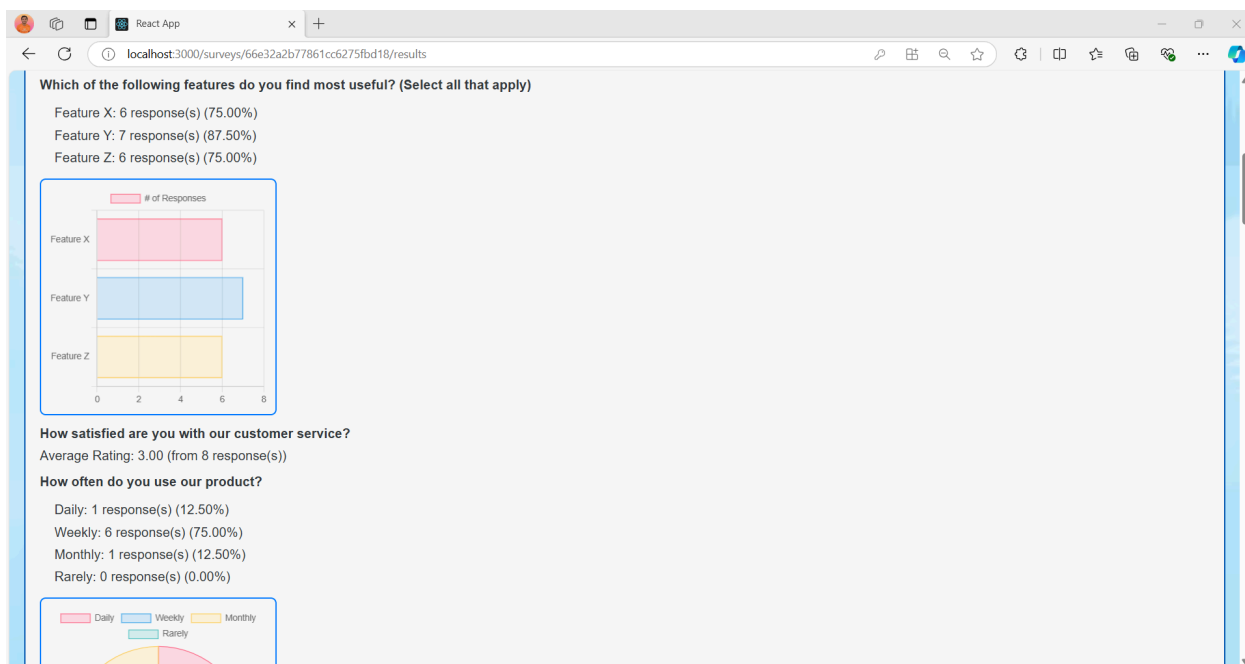


Fig 8.6.1 Survey Analysis



**Fig 8.6.2 Survey Analysis**

## 9. Conclusion

---

- The ***SurveyKaro.com*** project successfully achieved its goal of providing an efficient platform for:
  - Creating, managing, and responding to online surveys.
  - Implementing core functionalities such as user authentication using JWT, survey creation, editing, and deletion.
  - Allowing users to submit and retrieve survey responses effectively.
- **Backend API:**
  - Built with Node.js and Express.js to ensure smooth data handling.
- **Frontend Interface:**
  - Developed using React, providing a clean and intuitive user experience.
- **Database Solution:**
  - MongoDB was used for robust and flexible storage of user data, survey details, and responses.
- **System Capabilities:**
  - Successfully handles multiple users and surveys.
  - Ensures real-time response analysis.

## 10. Limitations and Future Extension

---

### Limitations :

- 1. Limited Question Types:** The survey creation feature supports basic question types like multiple choice and text inputs. However, more advanced question formats, such as ranking, or file uploads, are not yet available. This restricts the variety of data that can be collected.
- 2. Lack of Graphical Data Representation:** At present, the platform lacks the ability to visualize survey results through graphs or charts. This makes it harder for users to interpret data trends and analyze responses visually.

### Future Extensions :

- 1. Additional Question Types:** A key future enhancement will be to support a wider range of question types in surveys. This could include options like matrix-style questions, star ratings, and more customizable formats, giving creators more flexibility in collecting diverse forms of data.
- 2. Graphical Data Visualization:** Implementing graphical data representations will be a priority. Future versions of the platform could include bar charts, pie charts, and line graphs to help users visualize response trends, offering clearer insights from the data collected.

- 3. AI-Driven Insights for Subjective Responses:** A significant future extension would involve integrating AI to analyze subjective, text-based responses. Using natural language processing (NLP), the platform could automatically summarize common themes in open-ended answers and provide collective insights to survey creators. This would greatly enhance the value of subjective feedback by offering meaningful conclusions without requiring manual analysis.
  
- 4. Advanced Analytics and Reporting:** Future updates could introduce advanced analytics features, such as filtering results by demographics or timeframes, and exporting reports in various formats, allowing for deeper insights and more customizable reporting.

# 11. Bibliography

---

- 1. React Documentation** – Official documentation for React, used for building the frontend components of the application. <https://reactjs.org/docs>
- 2. Node.js Documentation** – Official Node.js documentation for server-side development and API handling. <https://nodejs.org/en/docs>
- 3. Express.js Documentation** – Reference for setting up the backend API using Express.js. <https://expressjs.com>
- 4. MongoDB Documentation** – Documentation for database management and querying using MongoDB. <https://www.mongodb.com/docs>
- 5. JWT Documentation** – JSON Web Tokens (JWT) used for secure user authentication. <https://jwt.io/introduction>
- 6. Postman Documentation** – Used for testing and verifying API calls. <https://learning.postman.com/docs/getting-started/introduction/>
- 7. GitHub** – Version control and collaboration platform used for project development and management. <https://github.com>
- 8 Chatgpt** – AI chatbot used for error resolving and enhancing code logic during the development phase. <https://chatgpt.com/>